

Hands-On-Learning: Baseline or Bust – Your First Reproducible Benchmark

Learning Objectives

By the end of this activity, learners will be able to:

- Run a lightweight benchmarking harness on a real 15-prompt evaluation dataset with fixed settings.
- Compute and interpret p50/p95 latency, tokens/sec, and cost per task using exported logs.
- Identify outliers and potential bottlenecks in LLM performance from baseline data.
- Formulate data-driven hypotheses for the biggest performance wins to pursue next.



Dataset Provided

Learners must have a real dataset, so here it is.

Dataset: support_chat_eval_v1.jsonl

```
{"id": 1, "prompt": "My order hasn't arrived yet. Can you check the status?"}  
{"id": 2, "prompt": "I forgot my password and can't log in. What should I do?"}  
{"id": 3, "prompt": "The app keeps crashing when I try to open it."}  
{"id": 4, "prompt": "How do I update my billing information?"}  
{"id": 5, "prompt": "Do you offer refunds for accidental purchases?"}  
{"id": 6, "prompt": "The live chat agent disconnected. Can you reconnect me?"}  
{"id": 7, "prompt": "I was double charged this month. Why?"}  
{"id": 8, "prompt": "How do I activate two-factor authentication?"}  
{"id": 9, "prompt": "Where can I download my previous invoices?"}
```

```
{"id": 10, "prompt": "The confirmation email never arrived."}  
{"id": 11, "prompt": "Is there a way to export my account data?"}  
{"id": 12, "prompt": "I need help cancelling my subscription."}  
{"id": 13, "prompt": "Why is the search function returning no results?"}  
{"id": 14, "prompt": "How do I change the language settings?"}  
{"id": 15, "prompt": "My payment keeps failing—what can I try?"}
```

A CSV version will also be included:
support_chat_eval_v1.csv

Structure of the Activity

Dataset Information / Resources Required

Evaluation dataset

You will use the provided dataset:

- datasets/support_chat_eval_v1.jsonl
- (or) datasets/support_chat_eval_v1.csv

Starter repository files

Provided in the project starter repo:

- run_eval.py
- config.yaml and .env.example
- runs/ folder for outputs

Tools Used

Primary language: Python 3.10+

Libraries:

→ openai (or a compatible LLM SDK)

→ pandas (analysis)

Other: Terminal, IDE (VS Code, PyCharm)

Setup Requirements

- API key in environment or config
- Python env with required packages installed
- Dataset file exists at:
 - datasets/support_chat_eval_v1.jsonl
- Ability to run Python scripts

Scenario

Activity 1: Run the Benchmark Harness

Objective

Execute the benchmarking harness on the provided dataset (15 prompts) and generate JSONL + CSV outputs.



Input/Data

datasets/support_chat_eval_v1.jsonl

This dataset contains real user-support prompts.

Steps to Complete

Step 1: Inspect the Harness

Open `run_eval.py`.

Locate the sections that:

- Load the dataset
- Call the LLM
- Write JSONL/CSV outputs

Step 2: Configure the Run

Ensure the script uses:

- The provided dataset
- A single model + region
- Deterministic settings (temperature 0) where possible

Step 3: Run the Baseline

```
python run_eval.py --dataset datasets/support_chat_eval_v1.jsonl  
--variant baseline
```

Confirm outputs appear in the runs/ directory.

Step 4: Verify Output Files

Check the summary CSV includes:

- prompt ID
- ttft_ms, total_ms
- token counts
- cost per task

Expected Outcome:

- A fully completed baseline run
- A CSV file with one row per test case

Activity 2: Analyze Baseline Metrics in pandas

Objective

Compute summary stats, outliers,
and build a baseline table.



Expected Outcome:

↳ CSV generated from Activity 1:
↳ runs/YYYY-MM-DD_baseline.csv

Essential imports

```
import pandas as pd  
df = pd.read_csv("runs/your_baseline_file.csv")
```

Steps to Complete

Step 1: Load Data into pandas

(unchanged)

Step 2: Inspect Raw Metrics

Confirm columns:

- ttft_ms
- total_ms
- input_tokens, output_tokens
- tokens_per_sec
- cost_per_task

Step 3: Compute Summary Stats

- p50
- p95
- avg tokens/sec
- avg cost/task

Step 4: Identify Outliers

- Sort by total_ms descending
- Optionally sort by cost_per_task
- Record 2–3 specific slow requests

Step 5: Build a Summary Table

Include:

p50/p95 ttft_ms

p50/p95 total_ms

avg tokens/sec

avg cost/task

outlier counts

Expected Outcome:

A clear summary table capturing the health of the baseline.

Activity 3: Formulate Performance Hypotheses

(unchanged except for dataset references; content was fine)

Compile Your Findings

1 Requirement

Clear description:

A reproducible baseline report containing raw metrics for each test case generated from Activity 1.

Include:

One CSV file with **one row per prompt**.

Columns for at least:

- prompt_id (or text snippet)
- ttft_ms
- total_ms
- input_tokens
- output_tokens
- tokens_per_sec (or derivable)
- cost_per_task (or derivable)

File should be produced by your run of:

```
python run_eval.py --dataset datasets/support_chat_eval_v1.jsonl  
--variant baseline
```

Ensure outputs are stored inside the runs/ directory with the correct timestamped filename.

2 Requirement

Clear description:

A structured summary table that aggregates key latency, throughput, and cost metrics across all test cases and highlights notable patterns.


```
graph LR; Root[ ] --> Include[Include:]; Root --> P50[p50 and p95 for:]; Root --> Avg[Average values for:]; Root --> Outlier[Outlier analysis including:]; Include --> P50; P50 --> TTFT[ttft_ms]; P50 --> Total[total_ms]; Avg --> Tokens[tokens_per_sec]; Avg --> Cost[cost_per_task]; Outlier --> Slowest[2-3 slowest requests (based on total_ms)];
```

Include:

p50 and p95 for:

ttft_ms

total_ms

Average values for:

tokens_per_sec

cost_per_task

Outlier analysis including:

2–3 slowest requests (based on total_ms)

Any unusually high-cost tasks

A brief narrative (2–4 sentences) summarizing what the numbers reveal about overall model behavior.

3 Requirement

Clear description:

A short narrative analysis connecting your baseline data to performance opportunities.

Include:

Two hypotheses about the largest performance wins available.

Each hypothesis must reference **specific metrics** (latency, tokens/sec, cost per task, or outliers).

Explain why each area looks promising for optimization.

The reflection should be approximately 200 words in total.

Summary

- **Provide a concise 3–5 sentence summary** synthesizing your baseline results:
- What you observed about latency distribution
- How stable throughput and token costs were
- Whether outliers indicate bottlenecks
- Overall readiness of the system for deeper optimization work
- This summary should serve as the executive snapshot of your findings.

Resources

Primary Materials:

- `run_eval.py`
- `datasets/support_chat_eval_v1.jsonl`
- `datasets/support_chat_eval_v1.csv`

Reference Docs:

- OpenAI API docs
- pandas docs

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.